HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

———————o0o———————

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

ĐỀ TÀI:

Variable-Length Records: Slotted Page Structure

Lớp : D23CQCE06-B

Nhóm : 09

Sinh viên thực hiện

1. Phùng Thu Hương - B23DCVT201
2. Chu Tuyết Nhi - B23DCCE075
3. Lê Như Quỳnh - B23DCCE081

Mục lục

1 Giải thích lý thuyết ngắn gọn 2
2 Cơ chế này giải quyết bài toán gì? 2
3 Nó hoạt động như thế nào? 3
4 Ưu điểm và nhược điểm là gì? 3
5 Khi nào nên dùng? 4
6 Tính toán thời gian để xử lý từng thao tác 4
7 Giải thích mã nguồn 5
7.1 Sinh dữ liệu (`1-data.py`) 5
7.2 Mã hóa bản ghi (`2-record.py`) 5
7.3 Cấu trúc dữ liệu trong trang (`3-page.py`) 6
7.4 Thuật toán chèn bản ghi 8
7.5 Thuật toán xóa bản ghi 8
7.6 Thuật toán compact 9
7.7 Đọc bản ghi 10
7.8 Các demo mô phỏng 10

1. Giải thích lý thuyết ngắn gọn

Cấu trúc Slotted Page (trang có khe) là một phương pháp tổ chức lưu trữ các bản ghi bên trong một trang đĩa. Trang này được chia làm hai phần di chuyển ngược chiều nhau: phần Header (lưu trữ số lượng bản ghi, con trỏ vùng trống, cùng với vị trí và kích thước của mỗi bản ghi) và phần không gian dữ liệu thực tế lưu trữ các bản ghi.

Bản ghi biến độ dài là các bản ghi không có kích thước vật lý cố định. Điều này đòi hỏi các cấu trúc quản lý dữ liệu phải sử dụng các cơ chế linh hoạt như mảng độ lệch hoặc các ký tự phân cách để nhận dạng và truy xuất.

Vì sao record biến độ dài xuất hiện trong thực tế: Trong cơ sở dữ liệu quan hệ, dù số lượng trường của một bảng là cố định, một bản ghi vẫn sẽ có độ dài thay đổi nếu nó chứa một hoặc nhiều trường có kích thước không xác định trước (như chuỗi ký tự độ dài tùy ý).

Vai trò của (offset, length): Cặp giá trị này thường được quản lý tại thư mục khe (slot directory) của trang hoặc phần đầu của bản ghi. Offset (độ lệch) đóng vai trò như một con trỏ chỉ điểm bắt đầu của bản ghi hoặc trường dữ liệu, còn length (độ dài) xác định kích thước. Ở một số biến thể, thư mục khe chỉ lưu offset, trong khi length được lưu ở những byte đầu tiên của chính bản ghi đó.

Vai trò của null bitmap: Null bitmap là một dãy bit (thường được lưu trữ gọn trong 1 byte hoặc hơn) dùng để ghi nhận trạng thái rỗng của các trường cụ thể trong bản ghi. Bitmap giúp hệ thống tối ưu hóa dung lượng lưu trữ do không cần phải cấp phát không gian vật lý thực tế cho các trường bị rỗng này.

Để lưu trữ và quản lý hiệu quả các bản ghi biến độ dài đó bên trong một trang đĩa, **cấu trúc Slotted Page (trang có khe)** được áp dụng. Trang này được chia làm hai phần di chuyển ngược chiều nhau: phần **Header** và phần **không gian dữ liệu thực tế**.

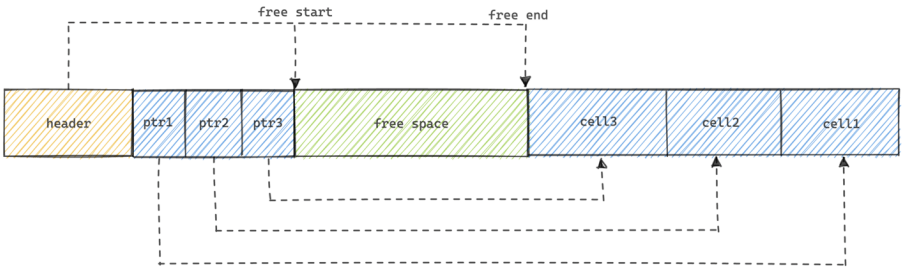
Trong Header có chứa một thư mục khe (slot directory) đóng vai trò quản lý các cặp giá trị **(offset, length)** cho từng bản ghi. *Offset* (độ lệch) hoạt động như một con trỏ chỉ điểm bắt đầu của bản ghi hoặc trường dữ liệu, còn *length* (độ dài) xác định kích thước. Ở một số biến thể, thư mục khe chỉ lưu offset, trong khi length được lưu ở những byte đầu tiên của chính bản ghi đó. Nhờ sự tách biệt này, dữ liệu thực tế có thể tự do phát triển hoặc co lại, dịch chuyển bên trong trang mà hệ thống vẫn quản lý được trọn vẹn.

2. Cơ chế này giải quyết bài toán gì?

- Quản lý các bản ghi có chiều dài thay đổi (Variable-length records) mà không làm lãng phí không gian bộ nhớ.
- Giải quyết vấn đề phân mảnh bên trong một trang: Khi xóa hoặc sửa một bản ghi, kích thước của nó thay đổi có thể tạo ra các khoảng trống (lỗ hổng), đòi hỏi phải dồn (compact) dữ liệu lại để vùng trống luôn liền mạch.

- Bảo toàn tham chiếu (địa chỉ): Khi các bản ghi bị dịch chuyển vật lý để gom vùng trống, các tham chiếu từ bên ngoài (từ index) trỏ vào bản ghi sẽ không bị hỏng vì chúng trỏ gián tiếp vào khe (slot) thay vì địa chỉ vật lý.

3. Nó hoạt động như thế nào?



- **Cấu trúc:** Slotted Page Header chứa số lượng bản ghi, con trỏ chỉ đến phần cuối của không gian trống và một mảng các khe (slot directory). Mỗi khe chứa độ lệch (offset/location) và kích thước (size) của một bản ghi. Header phát triển từ đầu trang xuống, còn các bản ghi phát triển từ cuối trang lên.

- **Định danh bản ghi (RID):** Các con trỏ ngoại vi không trỏ thẳng vào bản ghi mà trỏ vào khe tương ứng của nó trong Header (dạng `Page ID, Slot ID`).

- **Thêm bản ghi:** Hệ thống tìm trong Header xem có khe trống nào không, nếu có thì tái sử dụng; nếu không sẽ tạo thêm một khe ở cuối mảng khe. Dữ liệu bản ghi được ghi vào vùng trống và vị trí mới được cập nhật vào khe.

- **Xóa bản ghi:** Giá trị của khe được đánh dấu là trống (thường gán offset bằng -1). Các bản ghi nằm sau có thể được dịch chuyển vật lý lên để lấp đầy không gian trống, sau đó vị trí (offset) trong Header của các bản ghi bị dịch chuyển sẽ được tự động cập nhật.

4. Ưu điểm và nhược điểm là gì?

- **Ưu điểm:** Khắc phục triệt để sự phân mảnh không gian cục bộ bên trong trang. Bản ghi có thể tùy ý thay đổi vị trí vật lý mà không ảnh hưởng tới Record ID đã cấp phát ra ngoài, đảm bảo cấu trúc index bên ngoài không phải cập nhật lại. Phù hợp cho cả bản ghi kích thước thay đổi lẫn cố định.

- **Nhược điểm:** Tiêu tốn thêm không gian lưu trữ cho phần Header (offset và size của từng bản ghi). Tiêu tốn CPU khi phải dịch chuyển bản ghi liên tục trong quá trình xóa, dồn trang hoặc khi một bản ghi được cập nhật với kích thước lớn hơn vùng trống hiện tại.

5. Khi nào nên dùng?

- Sử dụng khi file chứa các bản ghi có chiều dài thay đổi (ví dụ: các trường kiểu `VARCHAR`, `BLOB` nhỏ, hoặc cấu trúc có trường lặp lại).
- Khi thiết kế cơ sở dữ liệu hỗ trợ nhiều định dạng bản ghi lưu chung trên một file (multitable clustering file) hoặc hệ thống có mật độ cập nhật/xóa bản ghi cao khiến việc dồn bộ nhớ là bắt buộc.
- Thực tế, cấu trúc này rất ưu việt và đang được sử dụng làm chuẩn lưu trữ cho Heap File và B+ Tree page trong đa số hệ quản trị cơ sở dữ liệu hiện đại (ví dụ PostgreSQL, DB2).

6. Tính toán thời gian để xử lý từng thao tác

Trong CSDL, thời gian xử lý được tính phần lớn dựa trên số lượt truy xuất đĩa (I/O Cost), với D là thời gian đọc/ghi 1 trang đĩa và C là chi phí CPU xử lý bản ghi trong bộ nhớ.

- **Đọc bản ghi (khi biết Record ID):** Chỉ cần 1 I/O (đọc trang lên bộ nhớ, mất thời gian D). Thời gian xử lý của CPU bằng $O(1)$ vì chỉ cần truy xuất trực tiếp vào khe (slot) thứ i trên Header để lấy địa chỉ vật lý của bản ghi.
- **Thêm bản ghi (Insert):** Mất 1 thao tác đọc trang lên (nếu chưa có trong buffer), cộng thêm chi phí CPU cập nhật Header và đẩy dữ liệu vào bộ nhớ, cuối cùng ghi trang xuống đĩa. Tổng thời gian tốn $2D + C$.
- **Xóa bản ghi (Delete):** Mất 1 thao tác đọc trang. Chi phí CPU sẽ phụ thuộc vào lượng dữ liệu phải dịch chuyển để dồn trang (compact). Sau đó mất thêm 1 thao tác ghi trang vừa dồn xuống đĩa. Tổng thời gian tốn $2D$ cộng với thời gian dịch chuyển mảng của CPU.

7. Giải thích mã nguồn

Phần này trình bày mã nguồn Python mô phỏng cơ chế Slotted Page. Mã nguồn gồm ba file chính: `1-data.py` (sinh dữ liệu), `2-record.py` (mã hóa bản ghi thành byte), và `3-page.py` (mô phỏng trang Slotted Page).

7.1. Sinh dữ liệu (`1-data.py`)

File này tạo ra ba tập dữ liệu dạng text: `students.txt` (100.000 dòng), `courses.txt` (100 dòng) và `enrollments.txt` (1.000.000 dòng). Mỗi dòng là một bản ghi CSV.

Listing 1: Sinh dữ liệu Student

```
NUM_STUDENTS = 100000

with open("students.txt", "w", encoding="utf-8") as f:
    for i in range(1, NUM_STUDENTS + 1):
        name = random_name()
        student = f"{i},{name},CNTT{random.randint(1,4)},"
                  f"{random_email(name, i)},{random_phone()}"
        f.write(student + "\n")
```

Mỗi Student có 5 trường: `student_id`, `full_name`, `class_name`, `email`, `phone`. Do `full_name` và `email` có độ dài thay đổi tùy vào tổ hợp họ tên được sinh ngẫu nhiên, nên mỗi bản ghi Student có kích thước khác nhau khi mã hóa thành byte. Đây là nguồn gốc tạo ra variable-length record trong bài mô phỏng.

Tương tự, Course có 4 trường và Enrollment có 4 trường. Ba bảng này dùng chung dataset cho toàn bộ demo.

7.2. Mã hóa bản ghi (`2-record.py`)

File này chỉ có hai hàm: encode biến danh sách trường thành byte, decode biến byte ngược lại thành danh sách.

Listing 2: Encode và decode bản ghi

```
def encode_record(fields: list) → bytes:
    str_fields = []
    for field in fields:
        if field is not None:
            str_fields.append(str(field))
        else:
            str_fields.append("")
    return ",".join(str_fields).encode("utf-8")
```

```python

def decode_record(raw: bytes) → list:
    return raw.decode("utf-8").split(",")
```

Các trường được nối bằng dấu phẩy rồi encode UTF-8. Trường `None` được thay bằng chuỗi rỗng. Cách này đơn giản nhưng đủ để minh họa bản ghi biến độ dài: mỗi bản ghi khi encode ra sẽ cho mảng byte có kích thước khác nhau tùy nội dung thực tế.

7.3. Cấu trúc dữ liệu trong trang (`3-page.py`)

Slot và BlockHeader

Listing 3: Cấu trúc Slot và BlockHeader

```python
@dataclass
class Slot:
    offset: int
    length: int

    @property
    def is_empty(self) → bool:
        return self.offset == 0xFFFF

@dataclass
class BlockHeader:
    num_entries: int
    free_space_ptr: int

    @property
    def header_size(self) → int:
        return 4 + self.num_entries * 4
```

`Slot` lưu cặp `(offset, length)` của một bản ghi. Khi `offset = 0xFFFF` (giá trị 65535), slot đó được xem là rỗng (đã bị xóa hoặc chưa dùng).

`BlockHeader` lưu hai giá trị:

- `num_entries`: tổng số slot đã cấp phát (bao gồm cả slot rỗng).
- `free_space_ptr`: vị trí byte bắt đầu của vùng trống, tính từ đầu trang.

Kích thước header được tính: 4 byte cho `num_entries` + `free_space_ptr`, cộng thêm 4× `num_entries` byte cho mảng slot (mỗi slot chiếm 4 byte: 2 byte offset + 2 byte length).

Khởi tạo trang

Listing 4: Khởi tạo SlottedPage

```python
class SlottedPage:
    def __init__(self, page_size: int = 4096):
        self.page_size = page_size
        self.data = bytearray(page_size)
        self._write_header(BlockHeader(0, page_size))
```

Trang được cấp phát là một `bytearray` có kích thước cố định (mặc định 4096 byte). Ban đầu `num_entries = 0` và `free_space_ptr = page_size`, nghĩa là toàn bộ trang đều là vùng trống. Header nằm ở đầu trang và phát triển xuống, dữ liệu bản ghi được ghi từ cuối trang và phát triển lên.

Đọc/ghi binary

Listing 5: Đọc ghi header và slot bằng struct

```python
def _read_header(self) → BlockHeader:
    num_entries, free_space_ptr = struct.unpack_from(">HH", self
        .data, 0)
    return BlockHeader(num_entries, free_space_ptr)

def _write_header(self, header: BlockHeader):
    struct.pack_into(">HH", self.data, 0,
                     header.num_entries, header.free_space_ptr)

def _read_slot(self, slot_id: int) → Slot:
    offset_pos = 4 + slot_id * 4
    offset, length = struct.unpack_from(">HH", self.data,
        offset_pos)
    return Slot(offset, length)

def _write_slot(self, slot_id: int, slot: Slot):
    offset_pos = 4 + slot_id * 4
    struct.pack_into(">HH", self.data, offset_pos,
                     0xFFFF if slot.is_empty else slot.offset,
                     slot.length)
```

Module `struct` của Python được dùng để đọc/ghi trực tiếp trên mảng byte, mô phỏng thao tác đọc/ghi ở mức vật lý trên trang đĩa. Format `">HH"` nghĩa là big-endian, hai số nguyên unsigned 16-bit (mỗi số 2 byte).

Header nằm tại vị trí byte 0. Slot thứ `i` nằm tại vị trí `4 + i * 4`. Cách bố trí này tương ứng trực tiếp với cấu trúc Slotted Page trong lý thuyết: Header ở đầu trang, slot directory nằm liền ngay sau.

7.4. Thuật toán chèn bản ghi

Listing 6: Hàm insert

```
def insert(self, record: bytes) → int:
    record_len = len(record)
    header = self._read_header()

    reuse_id = self._find_reusable_slot(header.num_entries)

    req_space = record_len if reuse_id != -1 else record_len + 4
    if self.free_space() < req_space:
        return -1

    new_offset = header.free_space_ptr - record_len
    self.data[new_offset : header.free_space_ptr] = record

    target_id = reuse_id if reuse_id != -1 else header.
       num_entries
    self._write_slot(target_id, Slot(new_offset, record_len))

    if reuse_id == -1:
        header.num_entries += 1

    header.free_space_ptr = new_offset
    self._write_header(header)
    return target_id
```

Quá trình chèn diễn ra theo các bước:

1. Duyệt mảng slot tìm slot rỗng (`offset = 0xFFFF`) để tái sử dụng. Nếu không tìm thấy, slot mới sẽ được thêm vào cuối mảng.
2. Tính không gian cần thiết. Nếu tái sử dụng slot cũ thì chỉ cần chỗ cho dữ liệu (`record_len`). Nếu phải tạo slot mới thì cần thêm 4 byte cho mục slot trong header.
3. Ghi dữ liệu vào ngay trước `free_space_ptr`. Dữ liệu phát triển từ cuối trang lên.
4. Cập nhật slot tương ứng với `(offset, length)` mới.
5. Đẩy `free_space_ptr` lên trên.

Hàm `_find_reusable_slot` duyệt tuyến tính qua mảng slot. Trường hợp xấu nhất là $O(n)$ với n là số slot hiện có.

7.5. Thuật toán xóa bản ghi

Listing 7: Hàm delete

```python
def delete(self, slot_id: int):
    header = self._read_header()
    if 0 ≤ slot_id < header.num_entries:
        slot = self._read_slot(slot_id)
        if not slot.is_empty:
            self._write_slot(slot_id, Slot(0xFFFF, 0))
```

Xóa bản ghi không di chuyển bất kỳ dữ liệu nào. Chỉ cần gán `offset = 0xFFFF` cho slot tương ứng. Dữ liệu cũ vẫn nằm trong vùng dữ liệu nhưng không còn truy xuất được qua slot directory. Thao tác này là $O(1)$.

Điểm quan trọng: slot không bị xóa khỏi mảng. `num_entries` không giảm. Slot vẫn giữ vị trí cũ trong header để bất kỳ con trỏ bên ngoài nào trỏ tới slot ID này vẫn hợp lệ (dù sẽ nhận được trạng thái rỗng khi truy vấn).

7.6. Thuật toán compact

Listing 8: Hàm compact

```python
def compact(self) → dict:
    moved_records = {}
    header = self._read_header()

    active_slots = []
    for i in range(header.num_entries):
        slot = self._read_slot(i)
        if not slot.is_empty:
            active_slots.append((i, slot))

    active_slots.sort(key=lambda item: item[1].offset,
                      reverse=True)

    new_free = self.page_size

    for slot_id, slot in active_slots:
        new_offset = new_free - slot.length
        if new_offset != slot.offset:
            record_data = self.data[
                slot.offset : slot.offset + slot.length]
            self.data[
                new_offset : new_offset + slot.length] =
                   record_data
            self._write_slot(slot_id,
                             Slot(new_offset, slot.length))
```

```
            moved_records[slot_id] = (slot.offset, new_offset)
        new_free = new_offset

    header.free_space_ptr = new_free
    self._write_header(header)
    return moved_records
```

Compact dồn tất cả bản ghi còn sống lại sát cuối trang, loại bỏ khoảng trống do xóa tạo ra. Các bước:

1. Thu thập tất cả slot chưa bị xóa.
2. Sắp xếp theo offset giảm dần. Bản ghi nằm gần cuối trang nhất được xử lý trước.
3. Duyệt lần lượt, dời từng bản ghi về sát cuối trang (hoặc sát bản ghi vừa dời trước đó).
4. Cập nhật `offset` trong slot directory để trỏ tới vị trí mới. `slot_id` không đổi.
5. Cập nhật `free_space_ptr` về vị trí mới sau khi dồn xong.

Đây là minh họa trực tiếp cho tính chất cốt lõi của Slotted Page: dữ liệu vật lý bị dời, nhưng slot ID trong header không thay đổi. Bất kỳ tham chiếu ngoài nào (từ index, từ trang khác) trỏ tới slot ID vẫn hoạt động đúng sau compact.

Độ phức tạp: $O(n \log n)$ do bước sắp xếp, với n là số bản ghi còn sống.

7.7. Đọc bản ghi

Listing 9: Hàm get

```
def get(self, slot_id: int) → bytes:
    header = self._read_header()
    if slot_id < 0 or slot_id ≥ header.num_entries:
        return None

    slot = self._read_slot(slot_id)
    if slot.is_empty:
        return None

    return self.data[slot.offset : slot.offset + slot.length]
```

Đọc bản ghi theo slot ID: kiểm tra biên, kiểm tra slot có rỗng không, rồi cắt đúng đoạn byte từ `offset` đến `offset + length`. Thao tác $O(1)$.

7.8. Các demo mô phỏng

Phần demo nằm trong thư mục `demo/logic/`. Các demo sử dụng trang kích thước 256 byte (thay vì 4096) để dễ quan sát.

Demo 1: Chèn record khác độ dài

Listing 10: Luồng chèn 4 bản ghi

```
records = build_demo_records()
page = SlottedPage(page_size=256)
slot_labels = {}

for key in ["course_a", "enrollment_a",
            "student_a", "student_b"]:
    seed = records[key]
    slot_id, elapsed_ms = insert_seed(
        page, slot_labels, seed)
```

Demo chèn lần lượt 4 bản ghi lấy từ dataset thực tế (1 Course, 1 Enrollment, 2 Student). Mỗi bản ghi có kích thước byte khác nhau. Sau mỗi lần chèn, trạng thái trang (header, slot directory, vùng dữ liệu) được ghi lại thành JSON để hiển thị trên giao diện web.

Demo 2: Xóa rồi compact

Listing 11: Xóa một record rồi compact

```
delete_id = inserted[1][1]
delete_ms = delete_slot(page, delete_id)

moved, compact_ms = compact_page(page)
```

Demo xóa bản ghi thứ hai (Enrollment) rồi gọi compact. Kết quả cho thấy:

- Sau xóa: slot vẫn tồn tại trong header nhưng với trạng thái rỗng. Vùng dữ liệu cũ vẫn còn nguyên (chưa bị ghi đè).
- Sau compact: các bản ghi còn sống được dồn sát cuối trang. Offset trong slot directory thay đổi, nhưng slot ID không đổi.

Demo 3: Slot giữ ổn định, dữ liệu dời

Demo này tập trung chỉ ra rằng sau compact, con trỏ ngoài (`Page ID, Slot ID`) vẫn trỏ đúng bản ghi mặc dù vị trí vật lý trên trang đã thay đổi. Đây là lý do tại sao pointer nên trỏ tới slot thay vì trỏ trực tiếp vào offset của record.

Listing 12: Ghi nhận bản ghi bị dời

```
moved, compact_ms = compact_page(page)
# moved la dict:
# {slot_id: (offset_cu, offset_moi), ...}
```

Dict `moved` ghi lại slot nào có bản ghi bị dời và dời từ đâu tới đâu. Các slot không có trong dict này là các slot không cần dời (đã ở đúng vị trí hoặc đã bị xóa).